# Ders 10 Çalışma Özeti

## Ders 10 Çalışma Özeti

### Genel Konular

- Problemi doğru anlamak
  - Algoritma tasarlamanın ilk adımı istenen problemi doğru anlamaktır. “Verilen sayının Armstrong sayısı olup olmadığını bulmak” ile “tüm üç basamaklı Armstrong sayılarını bulup saymak” farklı problemlerdir.
- Algoritma 6: Üç basamaklı Armstrong sayılarını bulan ve sayan algoritma (ekrandan, 00:33:50)
  - Dışarıdan girdi alınmaz. Kontrol edilecek sayılar (100-999) bellidir.
  - Adım sayısı belli olduğu için for döngüsü kullanılır.
  - Akış: START → `count = 0` → for `I ← 100`, adım `1`, son değer `999`:
    - `yuzler = I DIV 100`, `onlar = (I DIV 10) MOD 10`, `birler = I MOD 10`
    - `kup = yuzler*yuzler*yuzler + onlar*onlar*onlar + birler*birler*birler`
    - `I = kup` ise: `I` yazdırılır, `count = count + 1`
  - Döngüden sonra `count` yazdırılır → STOP.
  - Onlar basamağı `(I DIV 10) MOD 10` ile de bulunabilir.
  - Sayı bulunduğu anda ekrana yazdırılır, çünkü henüz saklanacak bir yer (dizi) yoktur.
  - For, while ve if birbirinin içinde serbestçe kullanılabilir: for içinde if, for içinde for, while içinde for gibi.
- Anlamlı değişken isimleri
  - `b, c, d, e` yerine `yuzler, onlar, birler, kup` gibi, sakladığı bilgiyi anlatan isimler kullanılmalıdır. Bu kural programlamada da geçerlidir.
- Algoritma 7: Arabanın saniyelik ve ortalama yakıt tüketimi (ekrandan, 01:24:30)
  - Problem: Benzin tüketimi her saniye bilgi olarak gelir. −1 değeri gelirse sürüş bitmiştir. Her saniyede o ana kadarki ortalama tüketim ve sürüş sonunda sürüş ortalaması yazdırılacaktır.
  - Sürüşün kaç saniye süreceği bilinmez. Bu yüzden for değil, while kullanılır.
  - İlk (hatalı) tasarım: döngü içinde önce değer okunup sonra hesap yapılırsa, kullanıcı −1 girdiğinde son turda −1 de hesaba katılır ve ekrana yanlış bir ortalama basılır.
  - Doğru tasarım: ilk değer döngüden önce okunur, yeni değer döngü gövdesinin sonunda okunur.
    - START → `count = 0`, `toplam = 0` → `'İlk tüketim değerini giriniz'` → giriş: `X` → `ort = X`
    - while `X >= 0`:
      - `count = count + 1`, `toplam = toplam + X`, `ort = toplam / count`
      - çıkış: `ort`
      - `'Tüketim değerini giriniz'` → giriş: `X`
    - Döngüden çıkınca: `'Sürüş ortalaması', ort` → STOP.
  - Analiz (5, 6, 7, −1): hoca analizi o anki `count = 1`, `ort = toplam / (count − 1)` sürümüyle yapar. Yukarıdaki `count = 0` sürümünde count 1→2→3, toplam 5→11→18, ort 5→5,5→6. −1 gelince döngü biter ve sürüş ortalaması 6 olarak yazılır.
  - `count` sayacı 0’dan başlatılırsa ortalamada `count - 1` kullanılmaz. Sayaç 1’den başlatılacaksa ortalama, sayaç artırılmadan önce hesaplanmalıdır.
  - `ort = X` ilk değeri, döngüye hiç girilmeyen durum içindir. Hoca önce “bir saniye çalışıp durduysa” der, sonra düzeltir (01:32:15): döngüye yalnız ilk değer −1 girilirse girilmez, o zaman ortalama −1 görünür ve arabanın çalışmadığı anlaşılır.
  - Koşul `X >= 0` ya da `X ≠ -1` olabilir. `X >= 0`, sensörün hatalı olarak başka bir negatif

değer üretmesi durumunda da döngüyü durdurur.
  - Hoca, bu basit algoritmanın araçlardaki ortalama yakıt tüketimi göstergesinin mantığıyla aynı olduğunu belirtir. Gerçek sistemde değer, borudan geçen yakıtı ölçen bir sensörden gelir.
  - C dilindeki do-while yapısında önce iş yapılır, sonra koşul kontrol edilir. Bu, "önce bir değer oku" ihtiyacını farklı biçimde çözer.
- Sabitler, değişkenler ve diziler (ekrandan, 01:54:40)
  - `a = 5` ifadesinde `a` değişken, `5` sabittir (constant). `x = 3.2` ifadesinde `x` kesirli sayı değişkeni, `3.2` kesirli sayı sabitidir.
  - Bir değişken yalnızca bir değer saklar. Aynı tipte birden fazla değeri saklamak için dizi (array) kullanılır.
  - Tek boyutlu dizi (one-dimensional array): elemanlar köşeli parantez içindeki indisle erişilir. İki boyutlu diziler matris olarak adlandırılır.
  - Örnek dizi: `[5, 1, 4, -1, 0, 8, 9]`, eleman sayısı (number of elements) 7.
  - C'de indisler 0'dan başlar: `dizi[0] = 5`, `dizi[6] = 9`. Genel olarak `dizi[0] ... dizi[N-1]`.
  - Akış diyagramlarında indis 1'den başlatılabilir: `dizi[1] ... dizi[N]`. Derste akış diyagramlarında bu kabul kullanılacaktır.
  - Akış diyagramında dizi tanımlanmaz. C'de ise tanımlanır, örneğin `int A[100];`.
- Dizilerin bellekte saklanması
  - Dizi elemanları bellekte kesintisiz (continuous) ve arka arkaya saklanır. Aralarına başka veri girmez.
  - Ayrı değişkenler (`a`, `x`) ise belleğin farklı yerlerinde olabilir.
  - Bir tam sayı (integer) bu mimaride 4 byte kaplıyorsa ve dizi `0x1000` adresinden başlıyorsa, elemanların adresleri şöyledir: `dizi[0]` → `0x1000`, `dizi[1]` → `0x1004`, `dizi[2]` → `0x1008`, `dizi[3]` → `0x100C`, `dizi[4]` → `0x1010`, `dizi[5]` → `0x1014`.
  - Adresler onaltılık sistemdedir (`0x` öneki). 0x1008'den sonra 4 eklenince 0x100C gelir (C = 12).
  - Değişkenlerin hangi adreste olduğu program derlenirken saklanır. Değişken kullanıldığında o adrese gidilir.
- Algoritma 8: Ortalamanın üstündeki öğrencilerin sırasını bulma (ekrandan, 02:13:40)
  - Problem: N kişilik sınıfta final notları açıklanmıştır. Ortalamanın altındakiler başarısız, ortalamaya eşit ve üstündekiler başarılıdır. Başarılı öğrencilerin kaçıncı sırada (dizideki indis) olduğu bulunacaktır.
  - Örnek veri: `[20, 100, 80, 70, 60, 50, 90]`, N = 7.
  - Akış:
    - START → giriş: `N` → for `I ← 1` … `N`: giriş `A[I]` (notlar diziye okunur).
    - `toplam = 0` → for `I ← 1` … `N`: `toplam = toplam + A[I]` → `ort = toplam / N`.
    - for `I ← 1` … `N`: `A[I] >= ort` ise `I` yazdırılır → STOP.
  - Koşul > değil >= olmalıdır, çünkü ortalamaya eşit olanlar da başarılıdır.
  - Ortalama döngü içinde değil, döngüden sonra bir kez hesaplanır.
  - Başarılı öğrenciler ortalama bilinmeden belirlenemez. Bu yüzden kontrol, ortalama hesaplandıktan sonra ayrı bir döngüde yapılır.
  - Okuma ile toplama aynı döngüde de yapılabilir. Bu, N kez daha az dönüş sağlar ve daha hızlıdır. Derste anlaşılırlık için ayrı yazılmıştır.
  - `N` yerine `ogrenciSayisi` gibi anlamlı bir isim kullanılabilir.
- Alıştırma: 365 günlük sıcaklıklarda en soğuk ve en sıcak gün
  - 365 günün sıcaklığı tek tek okunur (sayı belli olduğu için dışarıdan N alınmaz). Değerler kesirli ve negatif olabilir (örn. 33,5, −2,4).
  - Yaklaşım: ilk eleman hem maksimum hem minimum kabul edilir. Sonra diğer elemanlar sırayla karşılaştırılır. Daha büyük bir değer görülürse yeni maksimum, daha küçük görülürse yeni minimum olur. Hangi günde olduğu da saklanır.
  - Başlangıç değeri olarak dışarıdan bir sınır (ör. −100 ve 100) vermek risklidir. Değer aralığı bilinmeyen problemlerde bu sınırlar yanlış sonuç verebilir. Bu yüzden her zaman ilk eleman başlangıç kabul edilir.
  - Bilgisayar değerleri "görmez". Pozitif/negatif gibi gruplamalar yapamaz, her değeri tek tek karşılaştırması gerekir.
  - Çözüm bir sonraki derste yapılacaktır.

## Hocanın Özellikle Vurguladığı Kısımlar

- Adım sayısı belli değilse while kullanılır
  - "Adım sayısı belli değil ama…" diye for kullanılmaz. Belli değilse while kullanılır.
- Koşulları dikkatle yazın
  - `X ≠ 1` ile `X ≠ -1` gibi küçük hatalar programın tamamen yanlış çalışmasına yol açar.
- Akış diyagramında tanımlama yoktur
  - Değişkenler ve diziler tanımlanmadan, kullanıldıkları yerden itibaren kullanılabilir.
- Analizi hemen yapın
  - Algoritmayı çizdikten hemen sonra küçük bir örnekle analiz etmek hataları erken yakalar.
- Çözümü kendiniz çizin
  - Derste verilen süreler, problemin üzerine düşünüp çizmeniz içindir. "Böyle yapılır" deyip çizmemek öğrenmeyi engeller.
- Dizilerin başlangıç indisi
  - Akış diyagramlarında 1, C'de 0 kullanılır. Hangisini kullandığınız tutarlı olmalıdır.
- Maksimum ve minimum için ilk eleman
  - Başlangıç değeri olarak her zaman dizinin ilk elemanı kullanılmalıdır.

## Kısa Tekrar Notları

- Tüm Armstrong sayıları: for I = 100..999, basamaklar DIV/MOD ile, küpler toplamı = I ise yazdır ve say.
- Yakıt ortalaması: while (X >= 0). İlk değer döngüden önce, sonraki değerler gövdenin sonunda okunur.
- Sayaç 0'dan başlarsa ortalama = toplam / count.
- Değişken bir değer, dizi aynı tipte çok değer saklar.
- C: `dizi[0]..dizi[N-1]`. Akış diyagramı: `dizi[1]..dizi[N]`.
- Dizi elemanları bellekte arka arkayadır. int 4 byte ise adresler 4'er artar (0x1000, 0x1004, 0x1008, 0x100C…).
- Ortalamanın üstündekiler: oku → topla → ortalama → ayrı döngüde `A[I] >= ort` kontrolü.
- Maksimum/minimum: başlangıç = ilk eleman.

## Detaylı Açıklamalar

Ders, önceki derslerde öğrenilen döngülerle yeni problemler çözülerek başlar. İlk problem tüm üç basamaklı Armstrong sayılarını bulmak ve kaç tane olduklarını saymaktır. Hoca önce bu problemin daha önce çözülen "verilen sayı Armstrong sayısı mı" probleminden farklı olduğunu vurgular. Bu problemde kullanıcıdan girdi alınmaz, çünkü kontrol edilecek sayılar 100 ile 999 arasındadır. Adım sayısı belli olduğu için for döngüsü kullanılır. Her turda döngü indisi I'nın basamakları DIV ve MOD işlemleriyle ayrılır, küpler toplamı hesaplanır ve I'ya eşitse sayı yazdırılıp sayaç artırılır. Döngü bittikten sonra sayaç yazdırılır. Hoca bu örnek üzerinden değişkenlere sakladıkları bilgiyi anlatan isimler verilmesi gerektiğini hatırlatır.

İkinci problem gerçek hayattan bir örnektir: bir arabanın her saniye gelen yakıt tüketimi değerleriyle anlık ortalama ve sürüş ortalamasının hesaplanması. Sürüşün ne zaman biteceği bilinmediği için while kullanılır. $-1$ değeri sürüşün bittiğini gösterir. Öğrencilerle birlikte ilk tasarımın sorunu bulunur: değer döngünün başında okunursa son turda $-1$ de toplama katılır ve ekrana yanlış bir ortalama yazılır. Çözüm olarak ilk değer döngüden önce okunur, sonraki değerler ise döngü gövdesinin en sonunda okunur. Böylece $-1$ geldiğinde koşul kontrolü yapılır ve döngüden hesaplama yapılmadan çıkılır. Sayacın başlangıç değeri ile ortalama formülü arasındaki ilişki de tartışılır: sayaç 0'dan başlıyorsa ortalama toplam/sayaç, 1'den başlıyorsa işlem sırası değiştirilmelidir. Örnek analizde 5, 6, 7 değerlerinden sonra $-1$ girildiğinde sürüş ortalaması 6 bulunur.

Dersin ikinci yarısında dizilere giriş yapılır. Bir değişken tek bir değer saklarken, dizi aynı tipteki birden fazla değeri bir arada saklar. Dizi elemanlarına köşeli parantez içindeki indisle erişilir. C dilinde indisler 0'dan başlar ve N elemanlı bir dizinin son indisi $N-1$'dir. Akış diyagramlarında ise indislerin 1'den N'e kadar gittiği kabul edilir. Dizi elemanları bellekte kesintisiz biçimde art arda saklanır. Bir tam sayı 4 byte kaplıyorsa, ilk elemanı 0x1000 adresinde olan

dizinin sonraki elemanları 0x1004, 0x1008, 0x100C, 0x1010 adreslerinde bulunur. Ayrı ayrı tanımlanan değişkenler ise belleğin farklı bölgelerinde olabilir.

Dizilerle çözülen ilk problemde N öğrencinin final notları bir diziye okunur, ortalama hesaplanır ve ortalamaya eşit ya da üstünde olan öğrencilerin dizideki sırası yazdırılır. Çözüm üç döngüden oluşur: okuma, toplama ve kontrol. Ortalama yalnızca toplama döngüsü bittikten sonra bir kez hesaplanır. Başarılılık kontrolü ortalama bilinmeden yapılamayacağı için ayrı bir döngüde yapılır. Koşulun >= olması, ortalamaya eşit olanların da başarılı sayılması için gereklidir. Okuma ve toplama tek döngüde birleştirilebilir ve bu daha hızlı olur, ancak derste anlaşılırlık için ayrı gösterilmiştir.

Ders sonunda 365 günlük sıcaklık verisinden en sıcak ve en soğuk günü bulma problemi alıştırma olarak verilir. Hoca çözüm fikrini sözel olarak anlatır: ilk değer hem maksimum hem minimum kabul edilir, sonraki her değer bu değerlerle karşılaştırılır ve gerekirse güncellenir. Dışarıdan sabit bir başlangıç değeri vermek, değer aralığının bilinmediği durumlarda yanlış sonuç verebileceği için önerilmez. Bilgisayarın verileri insan gibi bir bakışta göremediği, her değeri tek tek karşılaştırması gerektiği vurgulanır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.